Policy Bank: Development-Selected Routing of Public Robot Policies for BEHAVIOR-1K

Team KANR technical report, 2026 BEHAVIOR Challenge

Keshav Balakrishna Abhijay Gangerapu
Rithik Satarla
Team KANR (Independent)

Project page: https://policy-bank.vercel.app
Code and data: https://github.com/FungousLand1941/policy-bank

October 2026

Abstract

The 2026 BEHAVIOR Challenge asks one system to complete 100 long-horizon household tasks in simulation. Rather than train a single generalist, we assemble a bank of ten vision-language-action checkpoints, nine public and one continued by us, and route each task to one of them. Routing uses development instances only: an empirical-Bayes model combines public reference scores, policy levels estimated against a reference policy, and our eligible development rollouts (800 in total, 724 available when the main route froze), and conservative switching rules guard each change of policy. The frozen route reaches $Q = 0.156$ (95% CI 0.143–0.168) on the ten public reporting instances of every task; 71 tasks score above zero and 37 of 1,000 episodes, in 15 tasks, succeed. Development estimates ranked tasks moderately well (Spearman 0.68) but were optimistic by 0.050 on average, and the gap sits in tasks whose estimate rested on a prior or a single rollout. A posterior-predictive simulation of our own procedure finds that, within the model, ranking by raw development means delivers a worse route and claims about 0.12 more than it delivers, while perfect knowledge of the candidates would add only about 0.04 on the new tasks. We release the route, all 1,000 metric files, the development ledger, the selector, the simulator and a fail-stop serving layer.

Values marked † come from pipeline logs and reports that are not part of the release. Every other number is recomputable from the released data; the generated ones come from the released code (`python -m policybank analyze`).

1 Introduction

BEHAVIOR-1K [15, 16] defines everyday household activities in realistic simulation, and its 2026 challenge [21] scores a single submission on 100 of them. The tasks are long: mean human demonstrations run from about one to fifteen minutes, and an episode ends when the goal is reached or when the robot has used 1.5 times the mean demonstration length. Partial credit counts the fraction of goal conditions satisfied, and the challenge score Q averages it over tasks.

By 2026 several strong public checkpoints existed, all from the $\pi_0/\pi_{0.5}$ vision-language-action family [6, 17, 18]: the Comet team's 2025 models [3], the four checkpoints of the 2025 winning solution [13], continuations of these on the 2026 demonstrations, and community specialists fine-tuned for a handful of tasks. Their quality differs sharply by task. With a fixed compute budget, we ask a practical question: *which checkpoint should run each task, decided from development data alone, without fooling ourselves with noise?*

This is per-instance algorithm selection [12, 19] with three complications. Evidence is scarce: a rollout runs for up to 22 simulated minutes and longer in wall-clock time, so most (task, policy) pairs get one to three development rollouts. Rollouts are noisy: the same policy on the same task scores 0 on one instance and 1 on the next. And picking the best-looking option from noisy estimates is optimistically biased [7, 20].

We therefore treat routing as shrinkage estimation [8]: every (task, policy) pair gets an informative prior, development rollouts update it, and conservative rules decide when a challenger may replace the incumbent.

The frozen route scored $Q = 0.156$ on the public reporting instances. More useful than the score are three findings about selection at this scale. First, the development estimates were optimistic, but the optimism sits where evidence was thinnest, not where selection was most aggressive. Second, in a posterior-predictive simulation of our own procedure, ranking by raw development means picks a worse route and overclaims by about 0.12, while the posterior picks a better route and claims it honestly. Third, within the model the candidate pool, not the selector, limits the score: an oracle choosing among the same candidates gains only about 0.04 on the new tasks.

Contributions.

1. A *policy bank* of ten checkpoints, each served behind its native adapter, and a frozen per-task route chosen without looking at reported instances (Sections 4–5).
2. A complete specification of the selector: priors with and without public reference results, conjugate updates, switching rules and hyperparameters (Algorithm 1, Table 2).
3. An audited evaluation: $Q = 0.156$ [0.143, 0.168] on 1,000 unchanged episodes, with score-blind duplicate handling and every deviation disclosed (Sections 6–7).
4. An analysis of calibration, a posterior-predictive simulation of the selection rules, and the statistical power of development comparisons (Section 8).
5. A fail-stop serving layer for hidden evaluation, with an adapter for OpenPI-compatible policy servers, and an open release of code, data and route (Section 9).

2 Related work

Vision-language-action policies. π_0 [6] attaches a flow-matching action expert to a PaliGemma vision-language backbone [5], and $\pi_{0.5}$ [18] adds co-training for open-world generalization. Every checkpoint in our bank descends from this line through OpenPI [17].

BEHAVIOR solutions. The 2025 winning solution [13] adapted $\pi_{0.5}$ with correlated-noise flow matching, stage tracking and task-specific correction rules, reaching $Q{\approx}0.26$ on 50 tasks. The Comet team's solution [3], second in 2025, studied pre-training and post-training recipes at scale. Apart from one continued-training run, we train nothing; our contribution is to select among such models and to measure the result carefully.

Selection under noise. Choosing the best of several noisy alternatives is the classical ranking-and-selection problem [4] and, sequentially, best-arm identification [11]. Empirical Bayes and Stein shrinkage [8] improve many noisy means at once by borrowing strength across them, and Tweedie's formula corrects the selection bias of reported winners [7]. Unadjusted estimates of selected options disappoint on average (the optimizer's curse [20]), and underpowered comparisons exaggerate effect sizes (Type M errors [9]). In learning research, reliable evaluation with few runs per task has been argued for in reinforcement learning [1]. We apply these ideas to routing robot policies under a fixed evaluation budget.

3 Problem setup

Tasks and scoring. Each of the 100 tasks t has a goal written in BDDL. An episode ends at success or when the simulator step count reaches 1.5 times the mean demonstration length for t. Its score $q \in [0, 1]$ is the fraction of goal conditions satisfied at the end, with $q = 1$ exactly at success. With $I = \{301, \ldots, 310\}$ the reporting instances,

$$Q = \frac{1}{100} \sum_{t=1}^{100} \frac{1}{|I|} \sum_{i \in I} q_{t,i}, \tag{1}$$

where a missing cell counts as zero. Instances 311–320 are reserved for development (our ledger contains rollouts on 311–319); the organizers evaluate submitted containers on hidden instances 321–340.

Table 1: The routed checkpoints. Revisions are Hugging Face commit prefixes. No weights are redistributed with this report.

Policy	Source (repository/subfolder)	Revision
Comet $\pi_{0.5}$ pt50	`sunshk/openpi_comet/pi05-b1kpt50-cs32`	`61739ff`
Comet $\pi_{0.5}$ pt12	`sunshk/openpi_comet/pi05-b1kpt12-cs32`	`61739ff`
pt50 + 2026 demos (ours)	`this work (continued training of pt50)`	`step 150997`
ymetz pt50 step 9999	`ymetz/pi05-b1k-pt50-steps-public/steps-cont-9999`	`20e80d2`
2025 winner ckpt 1	`IliaLarchenko/behavior_submission/checkpoint_1`	`89545bc`
2025 winner ckpt 2	`IliaLarchenko/behavior_submission/checkpoint_2`	`89545bc`
2025 winner ckpt 3	`IliaLarchenko/behavior_submission/checkpoint_3`	`89545bc`
2025 winner ckpt 4	`IliaLarchenko/behavior_submission/checkpoint_4`	`89545bc`
JackLiu 4-task specialist	`JackLiu0406/b1k-checkpoints/pi_behavior_100t_meta_ft_4task`	`ec1844b`
amyjia 5-task specialist	`amyjia1212/b1k-2026-t90-91-93-96-97/step_099999`	`7519e44`

Robot and interface. The robot is a wheeled bimanual R1Pro. A policy receives head and wrist RGB-D images, a 61-dimensional proprioceptive vector and the task identity, and returns 23-dimensional actions over a websocket, optionally several per query. Hidden evaluation requires an average of at least one simulator frame per second including inference, at most 600 s per query, and a single 24 GB GPU per policy[†].

Rules we imposed on ourselves. Policies use only onboard sensors at test time. Routing decisions use development instances only. A route is frozen and hashed before the reported runs of the affected tasks, and each reported cell is scored from its first completed run.

4 The policy bank

Table 1 lists the ten routed checkpoints with their exact sources. They fall into three families. **Comet** ($\pi_{0.5}$, replaying 32 actions per query): the released `pt50` and `pt12` checkpoints [3, 22]; a public continuation of `pt50` on the 2026 demonstrations by ymetz [23] (step 9999); and `s150997`, our own continued training of `pt50` on the 2026 demonstrations of all 100 tasks. **Winner** (replaying 20 actions per query): the four checkpoints of the 2025 winning solution [13, 14], served with their native wrapper and correction rules. **Winner-format specialists**: two community models fine-tuned for a few 2026 tasks [2, 10].

Every checkpoint runs behind its own native adapter: prompts, image preprocessing, normalization statistics, action representation and replay length are those of its publisher. The bank is a lookup table from task to checkpoint; we do not merge weights or ensemble actions.

Our development ledger holds 1,146 rollouts over 36 labels, including other checkpoint steps, a third-party 100-task model, action-search variants, time compression, gripper-correction add-ons and a modified robot configuration. A rollout is *eligible* for selection when it ran on a development instance with the default configuration, no evaluation-time variant and a routable policy label: 800 rollouts over 28 labels qualify.

5 Development-selected routing

5.1 Model

Let θ_{tp} be the expected score of policy p on task t. We place a Gaussian prior $\theta_{tp} \sim \mathcal{N}(\mu_{tp}, \tau_{tp}^2)$ and treat each development rollout as $y_{tpk} \sim \mathcal{N}(\theta_{tp}, \sigma_p^2)$. With n rollouts of mean $\bar{y}$, the posterior is

$$\theta_{tp} \mid y \sim \mathcal{N}\left(\frac{\mu_{tp}/\tau_{tp}^2 + n\bar{y}/\sigma_p^2}{1/\tau_{tp}^2 + n/\sigma_p^2},\ \left(\frac{1}{\tau_{tp}^2} + \frac{n}{\sigma_p^2}\right)^{-1} \right). \tag{2}$$

The rollout variance is the pooled within-task variance of p's development rollouts, shrunk toward a default: $\hat{\sigma}_p^2 = \max\{0.01,\ (\mathrm{SS}_p + \nu_0\sigma_0^2)/(\mathrm{df}_p + \nu_0)\}$ with $\sigma_0^2 = 0.08$ and $\nu_0 = 4$. Rollout scores are bounded and often

bimodal, so the Gaussian likelihood is an approximation whose job is to weigh a prior against a handful of rollouts. Section 8.2 checks it against Beta-distributed outcomes.

5.2 Priors where public reference results exist (tasks 0–49)

For the first 50 tasks, per-task results of the Comet and winner checkpoints from 2025 are public. They come from a different simulator setup and different instances, so we scale them by a transfer factor:

$$\mu_{tp} = \gamma_f \, r_{tp}, \qquad \tau_{tp}^2 = (r_{tp} \, s_\gamma)^2 + b^2, \tag{3}$$

where r_{tp} is the reference score, s_γ the factor's standard error and $b = 0.088$ a floor for task-specific deviations. For `pt50`, γ is the ratio of our development means to the reference over the other tasks (leave-one-task-out), giving $0.55 \pm 0.09^\dagger$. For the winner family, γ was fixed at $0.81 \pm 0.10^\dagger$ before stage 2 and used for its decisions; an empirical-Bayes estimate from an early subset of winner rollouts gave $0.69 \pm 0.11^\dagger$. `pt12`, which has no published per-task results of its own, borrows the Comet reference scores†.

5.3 Priors without reference results (tasks 50–99)

Tasks 50–99 are new in 2026, so no per-task results exist. We estimate a *policy level* instead. The reference policy `s150997` has eligible development rollouts on every task, so its level $\ell_{\mathrm{ref},-t}$ is the mean of its per-task means over the tasks other than t. A candidate's level is the reference level plus its mean paired difference on the other tasks both have run:

$$\ell_{p,-t} = \ell_{\mathrm{ref},-t} + \frac{1}{|S_{p,-t}|} \sum_{u \in S_{p,-t}} \big(\bar{y}_{up} - \bar{y}_{u,\mathrm{ref}}\big), \qquad S_{p,-t} = \{u \neq t : \text{ both ran } u\}. \tag{4}$$

Pairing removes much of the between-task variation. The level's standard error combines the reference level's error with the standard error of the mean paired difference, whose standard deviation is floored at 0.05 (and set to 0.25 when only one task is shared); this is conservative because it ignores their negative covariance. Levels are clipped at 0. Each level is combined with a weak hyperprior $\mathcal{N}(0.1, 0.1^2)$ and widened by a between-task standard deviation $\tau_b = 0.1$ to give (μ_{tp}, τ_{tp}).

Worked example. On `vacuuming_floors` (task 69), the released code gives the ymetz checkpoint a level of 0.179 ± 0.035, a prior of 0.170 ± 0.105 and, with rollout variance 0.091, a posterior of 0.152 ± 0.100 after its single development rollout scored 0. The original pipeline also pooled each level with the other step of the same training run: its candidate report of 12 October gave 0.184 ± 0.111 moving to $0.160^\dagger$ for the same rollout, and the frozen table of 13 October, built from more rollouts, records 0.166.

5.4 Switching rules

A posterior ranking alone would chase noise. On the 13 tasks from 0–49 that were routed in stage 2 (Section 5.5), the incumbent came from an earlier route and a challenger must pass every rule:

1. **Own evidence**: at least two of its own development rollouts on the task.
2. **Development guard**: it beats the incumbent on the development instances both ran, with at least two shared instances.
3. **Margin**: a posterior gain of at least 0.01 over the incumbent.
4. **Veto**: it is rejected if it scored 0 on every development instance where the incumbent scored above 0.1.

On tasks 50–99 the incumbent is the weak reference policy, so the highest eligible posterior wins if it clears the **margin**, subject to the **veto** and two further rules:

5. **Scope**: a specialist is a candidate only on tasks where it has its own development rollouts, and only with at least two rollouts in total.
6. **One checkpoint per training run**: when two steps of the same training run (here ymetz steps 6000 and 9999) win different tasks, every step is tried as the keeper, including one that won nowhere; the others are removed, all tasks are re-decided, and the keeper whose route has the highest total expected score stays. This keeps the number of serving images small; the reference is never removed.

The 37 stage-1 tasks had no incumbent. Their entries were written by an earlier version of the route writer, which ranked posteriors built from the same priors without the guard rules†.

Algorithm 1 Level-based routing for tasks without reference results (`select_by_level`)

Require: eligible development rollouts $\mathcal{D}$, reference r, generalists G, specialists S, training-run map F

$\hat{\sigma}_p^2 \leftarrow$ pooled, shrunk rollout variance of every p ▷ Sec. 5.1

function DECIDE(t, allowed)

  $C \leftarrow (G \cup \{s \in S : s \text{ ran } t,\ |\mathcal{D}_s| \geq 2\}) \cap$ allowed

  for $p \in C$ **do**

    $\ell \leftarrow \ell_{r,-t}$ if $p = r$ else $\ell_{p,-t}$ ▷ leave task t out

    prior $\leftarrow$ widen(combine($\ell, \mathcal{N}(0.1, 0.1^2)$), τ_b)

    $\text{post}_p \leftarrow$ Eq. (2) with p's rollouts on t

  drop every p vetoed on t

  $p^\star \leftarrow \arg\max_{p \neq r} \text{post}_p$

  return $p^\star$ if $\text{post}_{p^\star} - \text{post}_r \geq 0.01$ else r

allowed $\leftarrow G \cup S$; route $\leftarrow$ DECIDE(t, allowed) for all t

while some training run f has ≥ 2 members in route **do**

  for each member k of f **do** ▷ try every keeper

    $\text{trial}_k \leftarrow$ allowed $\setminus (f \setminus \{k\})$; $\text{value}_k \leftarrow \sum_t$ post of DECIDE(t, trial_k)

  allowed $\leftarrow \text{trial}_{\arg\max_k \text{value}_k}$; route $\leftarrow$ DECIDE(t, allowed) for all t

return route

Table 2: Selector hyperparameters (released implementation).

Quantity	Role	Value
σ_0^2, ν_0, floor	default rollout variance, pseudo-degrees of freedom, minimum	0.08, 4, 0.01
b	task-specific spread around a transferred reference score	0.088
$\mathcal{N}(0.1, 0.1^2)$	hyperprior on a policy level	–
τ_b	between-task standard deviation added to a level	0.1
paired-difference s.d. floor	minimum s.d. of paired differences	0.05
one shared task	s.d. of paired differences when one task is shared	0.25
single-task level s.d.	level s.d. when a policy ran one other task	0.1
level variance floor, clip	minimum variance of task means; lower bound of a level	10^{-4}, 0
transfer s.e. floor	minimum standard error of a transfer factor	0.001
scope	minimum total rollouts for a specialist to be a candidate	2
margin	minimum posterior gain to switch	0.01
own rollouts, shared instances	own-evidence and development-guard thresholds (tasks 0–49)	2, 2
veto threshold	incumbent score that makes a zero count against a challenger	0.1

5.5 Two-stage freeze

Evaluation compute was the binding constraint, so routing was frozen in two stages (Figure 1). Stage 1 fixed 37 tasks from 0–49 whose choice was already settled by priors and early development evidence; their reported runs began on 11 October. For 25 of them the routed policy had no own development rollouts on the task, so its expectation rested on its prior alone. The other 63 tasks (13 from 0–49 and all 50 from 50–99) were deferred while development rollouts accumulated. Their route was frozen at 12:54 UTC on 13 October from the pipeline's development table ($860^\dagger$ rollouts; the released ledger has 724 eligible rollouts finished before that time). Every reported episode of these tasks ran under its stage-2 entry. The original route tables are identified by SHA-256 in `data/route.json`, together with the canonical hash of the final route, and the test suite checks that every reported episode was produced by its task's routed policy.

6 Evaluation protocol

Infrastructure. Reported and development runs used TACC Stampede3 nodes with two NVIDIA RTX PRO 6000 Blackwell GPUs, running Isaac Sim 6.0.1 inside the organizers' container image. Isaac Sim 5.1, the version prescribed for hidden evaluation, crashed with the cluster's GPU driver; the organizers allowed Isaac Sim 6 for public results. Each simulator connects to its own policy server, because one native server

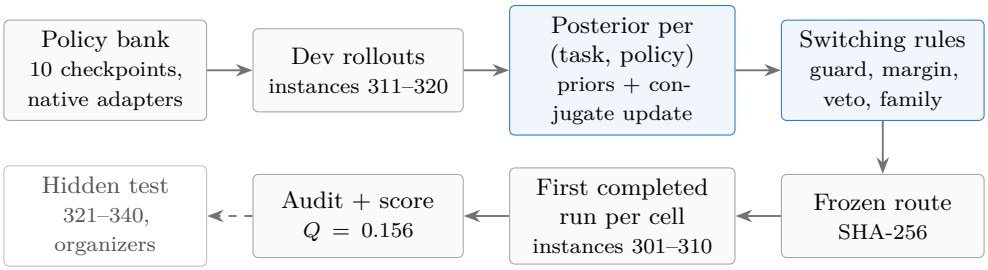


Figure 1: Development-selected routing. Reported instances (301–310) are never an input to a routing decision. Each stage's route is frozen and hashed before its reported runs.

Table 3: Headline results on the reporting instances (Isaac Sim 6.0.1). The confidence interval resamples instances within each task (10,000 draws, rescaled for ten instances); it does not cover the choice of route, a simulator-version shift or the hidden-instance draw. This is a self-evaluated public score, not a hidden-test score.

Challenge score Q (95% CI)	0.156 (0.143–0.168)
Episodes / tasks	1,000 / 100
Full successes	37 (3.7%) in 15 tasks
Tasks with any credit ($\bar{q} > 0$)	71
Tasks scoring zero on all ten instances	29
Episodes with $q > 0$	364

keeps a single action plan for all its connections. Camera inputs use the full-resolution RGB-D wrapper: a paired development comparison over 38† cells found 224-pixel inputs worse by 0.061† on average. Over the project we used roughly 1,500† documented allocated GPU-hours, including idle and failed allocations.

Integrity rules. The reporting set contains exactly one unchanged metrics file per (task, instance), 1,000 in total. All are released byte for byte in `data/metrics` with their SHA-256, and `policybank verify` re-checks any folder of evaluator files against them. Infrastructure retries produced two completed duplicate cells. In both we kept the attempt that *finished first*, regardless of its score: `spraying_for_bugs`/307 (kept 0.5, later 0.0) and `cook_brussels_sprouts`/303 (kept 0.0, later 0.0). The run logs† show that each later attempt started after the kept one ended. No development rerun or later experiment was merged into the reporting set.

Execution details. In the reporting set, 716 episodes ran with replay 32, 277 with replay 20, and 7 came from the farm script's recovery pass, which replays no actions (one model query per control step). We report those seven unchanged. The copy of the organizers' scorer bundled with the simulator divides by a 20-instance public pool and returns 0.078 on our files; the published reporting contract uses ten instances, and the same unmodified functions with that denominator give 0.15605. We changed neither the scorer nor the files.

7 Results

Table 3 summarizes the run. The score is mostly partial credit: 364 episodes satisfy at least one goal condition, but only 37 finish their task. All 963 unsuccessful episodes ran to the step cap; none ended early for another reason.

Figure 2 shows the per-task picture and Table 4 the ten highest tasks: three served by the four-task specialist, three by the ymetz checkpoint, two by Comet `pt50`, and one each by `pt12` and winner checkpoint 3. Twenty-nine tasks scored zero with their routed checkpoint. Their demonstrations are not longer than average (mean 319 s against 352 s over all tasks). On 23 of them no development rollout of any policy scored above zero. On four others the routed policy had itself succeeded at least once in development, which shows how little a few development rollouts say.

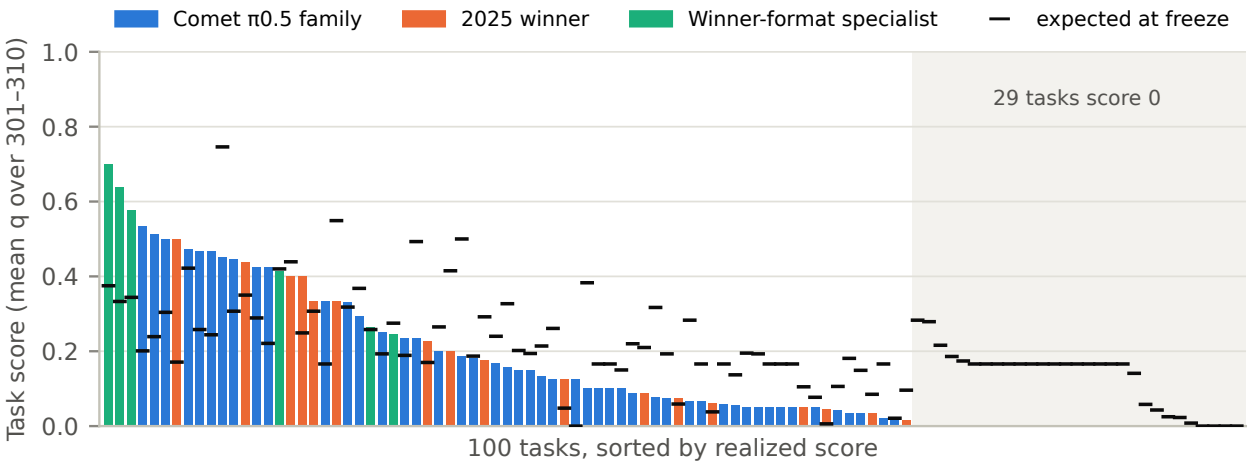


Figure 2: Realized score per task (bars, colored by policy family) against the posterior expectation at freeze (ticks), sorted by realized score. Twenty-nine tasks score zero on all ten instances. Per-task 95% intervals are listed in Appendix A.

Table 4: The ten highest-scoring tasks.

ID	Task	Policy	Expected	Realized	Successes
77	installing a modem	JackLiu 4-task specialist	0.375	0.700	6
54	putting away toys	JackLiu 4-task specialist	0.333	0.637	1
56	make rose centerpieces	JackLiu 4-task specialist	0.344	0.575	2
80	clean up broken glass	ymetz pt50 step 9999	0.201	0.533	2
24	storing food	Comet $\pi_{0.5}$ pt50	0.239	0.512	0
0	turning on radio	Comet $\pi_{0.5}$ pt12	0.304	0.500	5
31	clean boxing gloves	2025 winner ckpt 3	0.171	0.500	5
21	collecting childrens toys	Comet $\pi_{0.5}$ pt50	0.422	0.471	0
58	stacking wood	ymetz pt50 step 9999	0.244	0.467	2
87	collecting aluminum cans	ymetz pt50 step 9999	0.258	0.467	0

Per policy. Table 5 breaks the score down by policy. The ymetz continuation of `pt50` carries 42 of the 50 new tasks and contributes the most task points. Its per-task average (0.111) is modest, but it beat our own continued training on development pairs by +0.13 on average. The four-task specialist is the standout: 0.581 on its four tasks, eleven of the 37 successes, and well above its expectation of 0.368. The two `pt12` tasks and winner checkpoint 4's microwave-popcorn task also score high. Our own checkpoint `s150997` kept only three tasks and earned almost nothing on them, consistent with its low development level.

Per scene. Scores range from 0.30 on the five `hotel_suite_large` tasks to 0.075 on the ten upper-floor tasks of `house_double_floor` (Table 6). Mean demonstration length is only weakly related to task score (Spearman 0.11).

8 Analysis

8.1 Calibration

Figure 3 compares each task's posterior expectation at freeze with its realized score. Across tasks the ranking holds moderately well (Spearman 0.68, Pearson 0.60). Because each point is a different task, this mostly measures how well task difficulty was predicted; whether the right policy was chosen within a task needs the simulation of Section 8.2. Expectations were optimistic: the mean expectation was 0.206 and the mean realization 0.156.

Table 5: Per-policy outcome. "Expected" is the mean posterior expectation over the policy's routed tasks at freeze. "Task points" is the sum of its tasks' scores, so the column sums to $100\,Q$ up to rounding.

Policy	Family	Replay	Tasks	Expected	Realized	Successes	Task points
ymetz pt50 step 9999	Comet	32	42	0.188	0.111	5	4.65
Comet $\pi_{0.5}$ pt50	Comet	32	25	0.230	0.145	5	3.63
JackLiu 4-task specialist	Specialist	20	4	0.368	0.581	11	2.32
2025 winner ckpt 2	Winner	20	5	0.355	0.292	1	1.46
2025 winner ckpt 1	Winner	20	11	0.108	0.094	1	1.04
Comet $\pi_{0.5}$ pt12	Comet	32	2	0.305	0.472	5	0.94
2025 winner ckpt 3	Winner	20	5	0.137	0.119	5	0.59
amyjia 5-task specialist	Specialist	20	2	0.267	0.255	0	0.51
2025 winner ckpt 4	Winner	20	1	0.439	0.400	4	0.40
pt50 + 2026 demos (ours)	Comet	32	3	0.098	0.017	0	0.05

Table 6: Mean task score by scene.

Scene	Tasks	Mean task score
`hotel_suite_large`	5	0.302
`restaurant_diner`	8	0.198
`house_single_floor`	34	0.170
`Rs_int`	6	0.156
`house_double_floor_lower`	32	0.137
`office_cubicles_right`	5	0.128
`house_double_floor_upper`	10	0.075

Table 7 shows where the gap sits. Grouped by how many own development rollouts the routed policy had on the task when its route froze, the expectation overshot by 1.5 task points on the 25 tasks whose expectation rested on the prior alone and by 4.1 on the 46 tasks with one rollout, but undershot by 0.6 on the 29 tasks with two or more. Split by stage, the stage-1 tasks, whose priors are transfer priors, overshot by 2.7 task points and the new tasks 50–99, whose priors are policy levels, by 2.6, while the 13 stage-2 tasks from 0–49 were slightly pessimistic. Two policies illustrate the pattern. At freeze, the ymetz checkpoint's development mean on its 42 routed tasks (0.171) was no higher than on all 50 new tasks (0.176), so it was not chosen on lucky rollouts; but at freeze 39 of its 50 tasks had a single eligible rollout and 68% of its eligible rollouts used instance 313, and its tasks account for 3.2 of the 5.0 gap points. Conversely, the four-task specialist's raw development mean (0.64) was shrunk to an expectation of 0.37 and it delivered 0.58. The optimism therefore points at expectations built on thin, concentrated evidence, under both prior types, rather than at aggressive selection; the simulation below quantifies the selection part.

8.2 Simulating the selection procedure

The route can be scored only once, so we ask counterfactual questions with a model fitted to our own data (`policybank.simulate`). Each of 300 simulated worlds draws a true score θ_{tp} for every (task, candidate) cell on tasks 50–99 from the selector's posterior on the real ledger. Development rollouts are redrawn for the candidate cells of the real ledger with their counts (360 of the 411 eligible pre-freeze rollouts; rows of non-candidates are dropped), with true scores clipped to $[0, 1]$. Real episode scores are exactly zero far more often than a Beta distribution allows (64% of reported episodes, 74% of these development rollouts), and the veto rule reacts only to exact zeros, so each score is drawn from a zero-inflated Beta: zero with probability $(1-\theta)^c$, otherwise Beta with the mean that keeps the overall mean at θ. Least squares on the per-task zero fractions gives $c = 4.0$, and moments of the nonzero scores give concentration $\kappa = 6.42$; the fitted model reproduces the share of zeros. Five rules then pick a route from the simulated rollouts, and we record the true score of what each picked, the score it claimed, and one simulated reporting run of ten episodes per task. Because true scores are drawn from the selector's own posterior, its claims are close to calibrated by construction when the likelihood matches; the informative comparisons are between rules, and under an

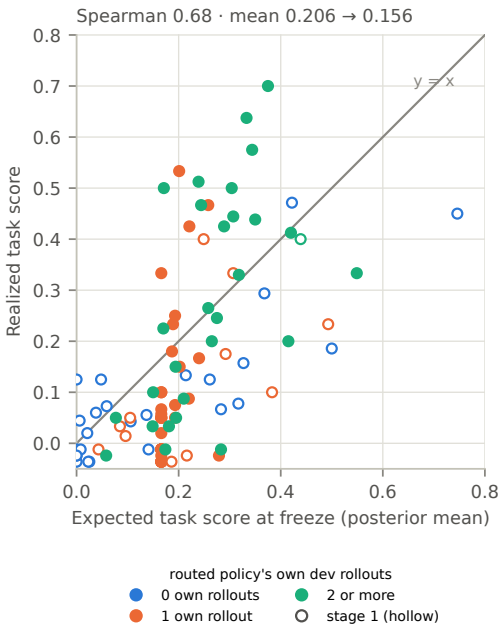


Figure 3: Expected (posterior at freeze) versus realized task score, colored by the routed policy's own development rollouts at freeze; hollow markers are stage-1 tasks. Zero-score tasks are jittered below 0.

Table 7: Calibration gap by evidence depth and by stage. Gap is expected minus realized, summed in task points.

Own dev rollouts	Tasks	Expected	Realized	Gap
0	25	0.162	0.100	+1.54
1	46	0.192	0.104	+4.07
2 or more	29	0.267	0.287	−0.58
Group	**Tasks**	**Expected**	**Realized**	**Gap**
Stage 1 (tasks 0–49)	37	0.188	0.115	+2.70
Stage 2, tasks 0–49	13	0.273	0.294	−0.27
Stage 2, tasks 50–99	50	0.203	0.151	+2.60

Table 8: Posterior-predictive study on tasks 50–99 (300 worlds). "Raw gap" is the chosen policies' raw development mean minus their true score, on tasks where they have rollouts. Regret is relative to the oracle.

Rule	True	90% range	Claimed	Optimism	Raw gap	Regret
Reference policy only	0.062	0.052–0.073	–	–	–	0.167
Highest raw development mean	0.182	0.158–0.204	0.299	+0.117	+0.117	0.047
Highest posterior, no rules	0.196	0.173–0.218	0.198	+0.002	+0.079	0.033
Strict rules (ours)	0.186	0.153–0.211	0.189	+0.002	+0.045	0.043
Oracle (true best candidate)	0.229	0.212–0.248	0.229	+0.000	–	0.000

outcome model that is not Gaussian.

Table 8 and Figure 4 give four results.

1. **Raw means choose worse and overclaim.** Within the model, choosing by raw development mean delivers 0.182 but claims 0.299 (optimism +0.117), while the posterior rules claim almost exactly what they deliver (optimism +0.002 for ours). With this many single, zero-heavy rollouts, shrinkage improves both the choice and the claim [7].
2. **The strict rules cost a little and cut the raw selection bias.** Within the model the rules give up 0.010 against the unconstrained posterior (0.186 versus 0.196), but the raw development means of the policies they choose overstate the truth by 0.045 rather than 0.079. Their value lies outside the model: broken ports, porting errors and outcome distributions the Gaussian misses.
3. **The candidate pool is the limit.** An oracle choosing among the same candidates reaches 0.229, only 0.043 above our rule on these 50 tasks (about 0.02 in Q). This gap is itself bounded by how strongly the posterior shrinks; the four-task specialist, which delivered 0.58 against an expectation of 0.37, shows the model can understate a strong candidate. Multiplying the development budget by eight raises our rule to 0.197 and raw-mean selection to 0.219: with plentiful evidence the conservative rules start to cost value.
4. **Our reported score is unlucky or the expectations were optimistic.** On these tasks the frozen route claimed 0.203 and reported 0.151. Among simulated reporting runs of our rule, 0.151 falls at

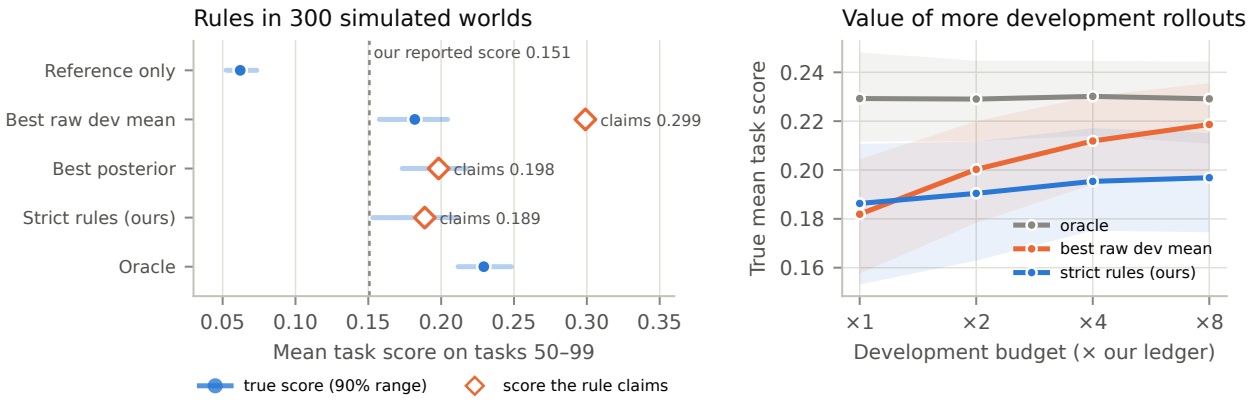


Figure 4: Left: true mean score of each rule's route (dot, 90% range across worlds) and the score it claims (diamond); the dashed line is our reported score on these tasks. Right: true score as the development budget is multiplied, with 90% bands.

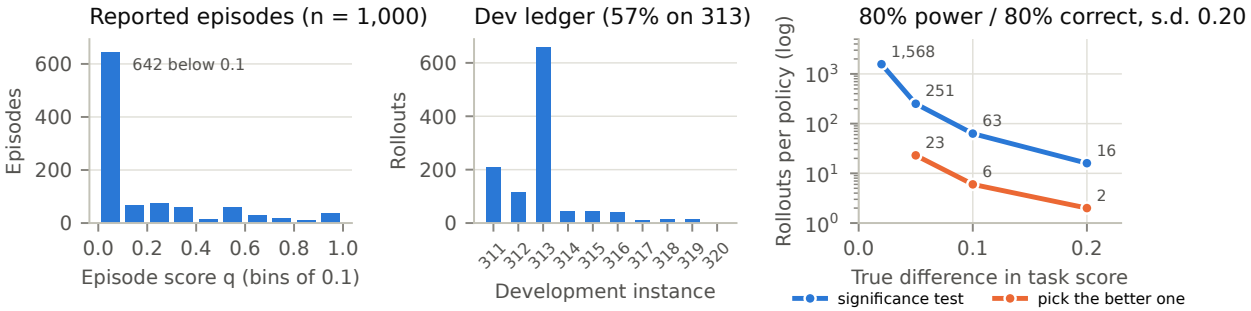


Figure 5: Left: episode scores pile up at zero. Middle: development rollouts concentrated on one instance (none on 320). Right: rollouts per policy needed to detect a difference with a two-sided test at 80% power ($\alpha = 0.05$, unpaired) and to pick the better of two policies with probability 0.8.

about the 4th percentile, just below the 90% range (0.153–0.214). Together with Section 8.1, the likelier explanation is that expectations built on mostly single rollouts on one instance were too high.

8.3 Rollout noise and statistical power

Within a task, reported episode scores have a standard deviation of 0.200 (0.237 excluding the 29 zero-score tasks); with one rollout per instance this mixes instance difficulty with rollout randomness. 642 of 1,000 episodes score below 0.1 (Figure 5). Two framings of power matter. A significance test that two policies differ by 0.05 needs about 251 rollouts per policy (63 for 0.10, 16 for 0.20); this bounds what can be *verified*. Picking the better of two policies with probability 0.8, which is what routing needs, takes about 23 rollouts per policy for a difference of 0.05, 6 for 0.10 and 2 for 0.20. Evaluating both policies on the same instances would lower both numbers further. Our eligible ledger falls short even of the selection numbers: of its 404 (task, policy) cells, 198 have a single rollout and 91% have at most three, and 47% of eligible rollouts (57% of all) used instance 313. Before the deadline we estimated that all 54† remaining per-task switch proposals together were worth about +0.007† in Q, too little to justify a rerun.

8.4 Reproducing the selection

The released selector (`policybank.selection`) re-derives the stage-2 decisions for tasks 50–99 from the released ledger, restricted to the 411 eligible rollouts on those tasks that finished before the freeze. It agrees with the frozen route on 48 of 50 tasks, including all 8 that route away from the ymetz checkpoint; routing ymetz everywhere would agree on 42. In both disagreements the reimplementation keeps the reference policy.

Task 70 is a near tie (posterior 0.176 against 0.173); on task 87 the reimplementation prefers the reference by 0.06 (0.288 against 0.229), where the frozen table had the ymetz checkpoint ahead by 0.035. The original pipeline used a slightly different development table and pooled the two ymetz steps. The 13 stage-2 tasks from 0–49, and all of stage 1, depend on third-party reference tables that we do not redistribute, so they are not reproduced.

8.5 What did not make the route

Several ideas were tested on development instances and rejected, or never reached a decisive comparison: replaying a recorded demonstration on development scenes that nearly duplicate demonstration scenes (0 of $14^{\dagger}$ such episodes succeeded, mostly from grasp misses), time-compressed action playback, gripper-correction add-ons for the Comet family, sampled action search, a modified robot configuration, 224-pixel camera inputs, and a third-party 100-task model whose port to our serving stack never produced enough nonzero development rollouts to rule out a porting error. Given the power analysis above, "rejected" often means "not shown to help" rather than "shown not to help".

9 Serving for hidden evaluation

Hidden evaluation runs the submitted containers on the organizers' hardware. We package one image per routed policy, and each container is bound at runtime to a single task. The released server enforces a strict contract on every frame and fails closed:

- one task per container and one evaluator session at a time (a second connection is refused with close code 1013); every new session resets the server's own action queues and the policy wrapper, and an upstream server that keeps per-episode state either receives a forwarded reset frame (opt-in) or must be restarted per session;
- batches of 1–10 environments with a fixed shape until reset; a reset clears the action queues before the next action;
- a single global transport chunk $K = 4$ that divides both native replay lengths (32 and 20), so the chunked action stream equals the native receding-horizon stream (tested for both lengths and for batches);
- malformed frames, wrong shapes, nonfinite actions, model exceptions and per-query timeouts (550 s, below the 600 s rule) close the connection with code 1011 and stop the server; there is no retry, no substitute action, no fallback checkpoint and no automatic restart, because a fabricated action could turn a fault into a silent wrong result. An evaluator that disconnects is not a policy fault; its session ends and the next one starts fresh. A failure of an upstream policy server, including its connection closing, is a policy fault.

Inference runs on a dedicated worker thread, and the command-line entry point leaves the process with `os._exit` after a failure, so a hung model cannot keep the container alive. `OpenPIRemotePolicy` puts the same contract in front of any OpenPI-compatible policy server, which is how the native checkpoints plug in. The release tests all of this against scripted and fake upstream policies. The release also includes a non-root container image for the server and a smoke test that exercises the full contract against it in continuous integration. In the original packaging, all ten native models passed† startup, warm-up and synthetic-input batch-1 and batch-10 protocol tests on an RTX 3090 as an unprivileged user; the longest batch-10 query took $3.38^{\dagger}$ s and sampled GPU memory stayed below $9^{\dagger}$ GB. A separate strict-isolation test, which restricted file writes and outbound connections, failed for all ten because CUDA reported no visible GPU†; we have not established whether the test harness or the images cause this. Full simulator episodes under Isaac Sim 5.1 at the required one frame per second on the organizers' GPU class have not been run, so hidden-evaluation readiness remains unverified.

10 Limitations and disclosures

- Q is a self-evaluated score on public reporting instances produced with Isaac Sim 6.0.1, not a hidden-test score or a prediction of placement. Hidden evaluation uses Isaac Sim 5.1, and simulator differences may shift scores. The bootstrap interval covers instance and rollout variation only.

- Development evidence is thin and concentrated on a few instances. Routes for individual tasks are uncertain even where the aggregate ranking is informative, and the calibration analysis suggests that expectations built on one or no own rollouts were too high under both prior types.
- The selector's Gaussian likelihood is an approximation for bounded, zero-heavy scores; the simulation checks it only against one alternative (zero-inflated Beta) outcome model, and its truths come from the selector's own posterior.
- Two duplicate cells were resolved by completion order, and seven episodes used a different replay length. Both are disclosed above and left unchanged.
- We do not redistribute model weights. The publishers of the winner checkpoints and the four-task specialist declare Apache-2.0 at the pinned revisions. The ymetz card defers to its base checkpoint's terms. We did not find an explicit licence for the separately hosted Comet weights or for the specialist used on tasks 96–97. Credits are attribution, not a licence grant.
- Values marked † come from pipeline logs and cannot be re-derived from the release.

11 Conclusion

A bank of public checkpoints, routed per task from development data alone, reached $Q = 0.156$ on the public reporting instances of the 2026 BEHAVIOR Challenge. Most of the score is partial credit, and 29 tasks scored zero with their routed checkpoint. The more transferable lessons concern measurement. At this scale per-rollout noise is large, a significance test cannot verify per-task gains below about 0.25 with ten rollouts per arm, and estimates built on one rollout are optimistic even after shrinkage. A frozen route and score-blind bookkeeping make the reported number honest; priors, conservative switching rules and a simulation of the procedure itself show how far to trust the route behind it. Code, data and the route are available at https://github.com/FungousLand1941/policy-bank.

Acknowledgments

We thank the authors of every checkpoint in the bank: Team Comet, Ilia Larchenko, Gleb Zarin and Akash Karnatak, ymetz, JackLiu0406 and amyjia1212. We also thank Physical Intelligence for OpenPI and Google for PaliGemma, the Stanford BEHAVIOR-1K team for the benchmark, the challenge and the demonstrations, and the Texas Advanced Computing Center for Stampede3.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[2] amyjia1212. b1k-2026-t90-91-93-96-97. https://huggingface.co/amyjia1212/b1k-2026-t90-91-93-96-97, 2026.

[3] Junjie Bai, Yu-Wei Chao, Qizhi Chen, Jinwei Gu, Moo Jin Kim, Zhaoshuo Li, Xuan Li, Tsung-Yi Lin, Ming-Yu Liu, Nic Ma, Kaichun Mo, Delin Qu, Shangkun Sun, Hongchi Xia, Fangyin Wei, and Xiaohui Zeng. Openpi Comet: Competition solution for 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.10071*, 2025.

[4] Robert E. Bechhofer. A single-sample multiple decision procedure for ranking means of normal populations with known variances. *The Annals of Mathematical Statistics*, 25(1):16–39, 1954.

[5] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, et al. PaliGemma: A versatile 3B VLM for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

[6] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[7] Bradley Efron. Tweedie's formula and selection bias. *Journal of the American Statistical Association*, 106(496): 1602–1614, 2011.

[8] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311–319, 1975.

[9] Andrew Gelman and John Carlin. Beyond power calculations: Assessing type S (sign) and type M (magnitude) errors. *Perspectives on Psychological Science*, 9(6):641–651, 2014.

[10] JackLiu0406. b1k-checkpoints. https://huggingface.co/JackLiu0406/b1k-checkpoints, 2026.

[11] Emilie Kaufmann, Olivier Cappé, and Aurélien Garivier. On the complexity of best-arm identification in multi-armed bandit models. *Journal of Machine Learning Research*, 17(1):1–42, 2016.

[12] Pascal Kerschke, Holger H. Hoos, Frank Neumann, and Heike Trautmann. Automated algorithm selection: Survey and perspectives. *Evolutionary Computation*, 27(1):3–45, 2019.

[13] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. Task adaptation of vision-language-action model: 1st place solution for the 2025 BEHAVIOR challenge. *arXiv preprint arXiv:2512.06951*, 2025.

[14] Ilia Larchenko, Gleb Zarin, and Akash Karnatak. behavior_submission: 2025 BEHAVIOR challenge winning checkpoints. https://huggingface.co/IliaLarchenko/behavior_submission, 2025.

[15] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, et al. BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In *Proceedings of The 6th Conference on Robot Learning (CoRL 2022)*, volume 205 of *Proceedings of Machine Learning Research*, pages 80–93, 2023.

[16] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, et al. BEHAVIOR-1K: A human-centered, embodied AI benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024.

[17] Physical Intelligence. openpi: Open-source models and code for robot learning. https://github.com/Physical-Intelligence/openpi, 2025.

[18] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.

[19] John R. Rice. The algorithm selection problem. In *Advances in Computers*, volume 15, pages 65–118. Elsevier, 1976.

[20] James E. Smith and Robert L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[21] Stanford Vision and Learning Lab. The 2026 BEHAVIOR challenge. https://behavior.stanford.edu/challenge/, 2026. Rules, evaluation protocol and submission guidelines; accessed October 2026.

[22] Team Comet. openpi_comet model weights. https://huggingface.co/sunshk/openpi_comet, 2025.

[23] ymetz. pi05-b1k-pt50-steps-public: Comet pt50 continued on the 2026 demonstrations. https://huggingface.co/ymetz/pi05-b1k-pt50-steps-public, 2026.

A All tasks

Table 9: Every task: routed policy, stage, the routed policy's own development rollouts at freeze, posterior expectation at freeze, realized score over instances 301–310 with a bootstrap 95% interval, and full successes.

ID	Task	Policy	Stage	Own dev	Expected	Realized	95% CI	Succ.
0	turning on radio	Comet $\pi_{0.5}$ pt12	2	6	0.304	0.500	0.20–0.80	5
1	picking up trash	2025 winner ckpt 2	2	5	0.549	0.333	0.13–0.57	1
2	putting away Halloween decorations	Comet $\pi_{0.5}$ pt50	1	0	0.500	0.186	0.11–0.27	0
3	cleaning up plates and food	Comet $\pi_{0.5}$ pt50	1	0	0.106	0.043	0.00–0.09	0
4	can meat	2025 winner ckpt 3	1	0	0.006	0.044	0.00–0.09	0
5	setting mousetraps	2025 winner ckpt 1	1	1	0.307	0.333	0.20–0.50	1
6	hiding Easter eggs	Comet $\pi_{0.5}$ pt50	1	0	0.317	0.078	0.00–0.18	0
7	picking up toys	Comet $\pi_{0.5}$ pt50	2	5	0.265	0.200	0.05–0.37	0
8	rearranging kitchen furniture	Comet $\pi_{0.5}$ pt50	2	5	0.195	0.050	0.00–0.12	0
9	putting up Christmas decorations inside	Comet $\pi_{0.5}$ pt12	2	6	0.307	0.444	0.30–0.60	0
10	set up a coffee station in your kitchen	Comet $\pi_{0.5}$ pt50	1	0	0.214	0.133	0.08–0.17	0
11	putting dishes away after cleaning	Comet $\pi_{0.5}$ pt50	1	0	0.327	0.157	0.06–0.27	0
12	preparing lunch box	2025 winner ckpt 2	1	1	0.249	0.400	0.22–0.55	0
13	loading the car	2025 winner ckpt 1	1	1	0.085	0.033	0.00–0.10	0
14	carrying in groceries	2025 winner ckpt 1	2	6	0.170	0.225	0.15–0.30	0
15	bringing in wood	Comet $\pi_{0.5}$ pt50	1	0	0.283	0.067	0.00–0.17	0
16	moving boxes to storage	Comet $\pi_{0.5}$ pt50	1	0	0.746	0.450	0.20–0.70	3
17	bringing water	Comet $\pi_{0.5}$ pt50	1	1	0.493	0.233	0.03–0.47	1
18	tidying bedroom	2025 winner ckpt 2	2	6	0.415	0.200	0.07–0.33	0
19	outfit a basic toolbox	2025 winner ckpt 1	1	1	0.096	0.014	0.00–0.04	0
20	sorting vegetables	2025 winner ckpt 2	2	6	0.350	0.439	0.35–0.54	0
21	collecting childrens toys	Comet $\pi_{0.5}$ pt50	1	0	0.422	0.471	0.31–0.63	0
22	putting shoes on rack	Comet $\pi_{0.5}$ pt50	2	6	0.318	0.330	0.19–0.48	0
23	boxing books up for storage	Comet $\pi_{0.5}$ pt50	1	0	0.025	0.000	0.00–0.00	0
24	storing food	Comet $\pi_{0.5}$ pt50	2	5	0.239	0.512	0.33–0.66	0
25	clearing food from table into fridge	2025 winner ckpt 1	1	0	0.038	0.060	0.00–0.12	0
26	assembling gift baskets	Comet $\pi_{0.5}$ pt50	1	0	0.368	0.294	0.19–0.39	0
27	sorting household items	Comet $\pi_{0.5}$ pt50	1	0	0.261	0.125	0.04–0.24	0
28	getting organized for work	Comet $\pi_{0.5}$ pt50	1	0	0.021	0.020	0.00–0.05	0
29	clean up your desk	2025 winner ckpt 1	1	0	0.059	0.073	0.02–0.14	0
30	setting the fire	2025 winner ckpt 2	2	6	0.210	0.087	0.04–0.15	0
31	clean boxing gloves	2025 winner ckpt 3	2	6	0.171	0.500	0.20–0.80	5
32	wash a baseball cap	2025 winner ckpt 3	1	1	0.186	0.000	0.00–0.00	0
33	wash dog toys	pt50 + 2026 demos (ours)	1	1	0.043	0.000	0.00–0.00	0
34	hanging pictures	2025 winner ckpt 1	2	5	0.058	0.000	0.00–0.00	0
35	attach a camera to a tripod	Comet $\pi_{0.5}$ pt50	1	0	0.000	0.000	0.00–0.00	0
36	clean a patio	Comet $\pi_{0.5}$ pt50	1	0	0.000	0.000	0.00–0.00	0
37	clean a trumpet	Comet $\pi_{0.5}$ pt50	1	0	0.000	0.000	0.00–0.00	0
38	spraying for bugs	2025 winner ckpt 3	1	1	0.105	0.050	0.00–0.15	0
39	spraying fruit trees	Comet $\pi_{0.5}$ pt50	1	0	0.141	0.000	0.00–0.00	0
40	make microwave popcorn	2025 winner ckpt 4	1	2	0.439	0.400	0.10–0.70	4
41	cook cabbage	Comet $\pi_{0.5}$ pt50	1	0	0.000	0.125	0.00–0.25	0
42	chop an onion	2025 winner ckpt 1	1	1	0.292	0.175	0.07–0.30	0
43	slicing vegetables	Comet $\pi_{0.5}$ pt50	1	0	0.137	0.056	0.00–0.13	0
44	chopping wood	2025 winner ckpt 1	1	0	0.048	0.125	0.00–0.25	0
45	cook hot dogs	Comet $\pi_{0.5}$ pt50	1	1	0.383	0.100	0.00–0.30	1
46	cook bacon	2025 winner ckpt 3	1	1	0.216	0.000	0.00–0.00	0
47	freeze pies	2025 winner ckpt 1	1	0	0.023	0.000	0.00–0.00	0
48	canning food	2025 winner ckpt 1	1	0	0.008	0.000	0.00–0.00	0
49	make pizza	Comet $\pi_{0.5}$ pt50	1	0	0.000	0.000	0.00–0.00	0
50	freeze fruit	ymetz pt50 step 9999	2	1	0.166	0.057	0.00–0.14	0
51	cook a brisket	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
52	sorting bottles cans and paper	ymetz pt50 step 9999	2	1	0.166	0.100	0.06–0.14	0
53	tidying living room	ymetz pt50 step 9999	2	1	0.193	0.050	0.00–0.12	0
54	putting away toys	JackLiu 4-task specialist	2	2	0.333	0.637	0.46–0.79	1
55	re shelving library books	ymetz pt50 step 9999	2	1	0.240	0.167	0.03–0.30	0
56	make rose centerpieces	JackLiu 4-task specialist	2	3	0.344	0.575	0.38–0.75	2
57	sweeping garage	pt50 + 2026 demos (ours)	2	3	0.174	0.000	0.00–0.00	0
58	stacking wood	ymetz pt50 step 9999	2	4	0.244	0.467	0.27–0.67	2
59	organizing art supplies	ymetz pt50 step 9999	2	1	0.187	0.180	0.08–0.30	0
60	scrubbing bathroom floor	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
61	bringing paper to recycling	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
62	halve an egg	ymetz pt50 step 9999	2	1	0.166	0.020	0.00–0.06	0
63	installing smoke detectors	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
64	setting the table	ymetz pt50 step 9999	2	1	0.220	0.087	0.00–0.20	0
65	unloading the car	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
66	turning out all lights before sleep	ymetz pt50 step 9999	2	2	0.150	0.100	0.02–0.18	0
67	boxing food after dinner	pt50 + 2026 demos (ours)	2	3	0.077	0.050	0.00–0.10	0
68	cleaning up branches and twigs	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
69	vacuuming floors	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
70	thawing frozen food	ymetz pt50 step 9999	2	1	0.189	0.233	0.18–0.30	0

ID	Task	Policy	Stage	Own dev	Expected	Realized	95% CI	Succ.
71	clean your rusty garden tools	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
72	cook a frozen pie	ymetz pt50 step 9999	2	1	0.166	0.050	0.00–0.15	0
73	organizing school stuff	ymetz pt50 step 9999	2	3	0.194	0.150	0.07–0.23	0
74	carrying out garden furniture	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
75	put together a basic pruning kit	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
76	dispose of glass	ymetz pt50 step 9999	2	1	0.221	0.425	0.25–0.62	1
77	installing a modem	JackLiu 4-task specialist	2	3	0.375	0.700	0.45–0.95	6
78	make cabinet doors	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
79	polishing shoes	ymetz pt50 step 9999	2	3	0.181	0.033	0.00–0.08	0
80	clean up broken glass	ymetz pt50 step 9999	2	1	0.201	0.533	0.33–0.73	2
81	packing meal for delivery	ymetz pt50 step 9999	2	1	0.166	0.067	0.00–0.15	0
82	store batteries	ymetz pt50 step 9999	2	2	0.149	0.033	0.00–0.10	0
83	store honey	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
84	tidying bathroom	ymetz pt50 step 9999	2	1	0.193	0.250	0.15–0.35	0
85	putting dirty dishes in sink	ymetz pt50 step 9999	2	3	0.289	0.425	0.30–0.53	0
86	make gift bags for baby showers	ymetz pt50 step 9999	2	1	0.202	0.150	0.10–0.20	0
87	collecting aluminum cans	ymetz pt50 step 9999	2	1	0.258	0.467	0.27–0.65	0
88	rearrange your room	ymetz pt50 step 9999	2	1	0.166	0.333	0.20–0.47	0
89	installing a fax machine	ymetz pt50 step 9999	2	1	0.166	0.100	0.00–0.25	0
90	composting waste	ymetz pt50 step 9999	2	1	0.166	0.050	0.00–0.15	0
91	store produce	ymetz pt50 step 9999	2	1	0.279	0.000	0.00–0.00	0
92	installing a scanner	ymetz pt50 step 9999	2	1	0.166	0.050	0.00–0.15	0
93	clean a keyboard	ymetz pt50 step 9999	2	4	0.283	0.000	0.00–0.00	0
94	dispose of batteries	ymetz pt50 step 9999	2	1	0.193	0.075	0.00–0.15	0
95	cook brussels sprouts	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0
96	cook broccolini	amyjia 5-task specialist	2	3	0.275	0.245	0.07–0.46	0
97	setup a bar for a cocktail party	amyjia 5-task specialist	2	2	0.258	0.265	0.18–0.35	0
98	laying tile floors	JackLiu 4-task specialist	2	3	0.420	0.412	0.16–0.65	2
99	sorting books on shelf	ymetz pt50 step 9999	2	1	0.166	0.000	0.00–0.00	0